

VantaDB

PRD técnico, roadmap de 90 días y backlog priorizado

Documento de dirección de producto, arquitectura y ejecución

Versión	1.0
Fecha	12 de abril de 2026
Estado	Documento de trabajo y decisión
Producto	VantaDB
Propósito	Consolidar el PRD técnico, el roadmap de 90 días y el backlog priorizado en un único entregable ejecutable.

Base documental: consolidado de los informes estratégicos previos, la auditoría del repositorio y los documentos de requerimientos ya definidos.

1. Resumen ejecutivo

VantaDB tiene una tesis técnica valiosa, pero hoy su mayor riesgo no es un algoritmo puntual ni una dependencia aislada. El riesgo dominante es la desalineación entre visión, alcance, arquitectura, herramientas, narrativa y ritmo de ejecución.

En este momento el proyecto intenta abarcar demasiadas capas al mismo tiempo: motor embebido, modo servidor, memoria para agentes, base multimodelo general, lenguaje de consulta propio, integración Python, despliegue con Docker y aspiración enterprise. Esa combinación no solo es ambiciosa; también es operativamente costosa y estratégicamente difusa.

La decisión correcta para esta etapa es concentrar VantaDB en un producto inicial mucho más preciso:

VantaDB debe iniciar como un núcleo embebido y local-first de memoria persistente y retrieval híbrido para agentes, herramientas locales y aplicaciones de conocimiento.

Esa definición permite:

- una propuesta de valor entendible,
- una arquitectura defendible,
- una ruta realista de ejecución,
- y una expansión posterior a server mode y enterprise sin romper el núcleo.

La línea estratégica de este documento es inequívoca:

1. ****No construir una base de datos universal desde el día uno.****
2. ****No añadir otra base de datos al stack.****
3. ****Sí añadir los índices y capas que faltan para que el producto sea coherente.****
4. ****Mover el núcleo hacia Fjall como backend principal.****
5. ****Dejar RocksDB como backend opcional o referencia de benchmark/compatibilidad.****
6. ****Tratar todos los índices como materializaciones derivadas y reconstruibles.****
7. ****Definir y proteger desde ya el protocolo de coherencia entre documento, embedding, metadatos, filtros e índices.****

2. Declaración de producto

2.1 Qué es VantaDB

VantaDB es un motor embebido de memoria persistente, retrieval híbrido y almacenamiento estructurado de conocimiento, diseñado para:

- agentes de IA,
- automatizaciones locales,
- asistentes privados,
- herramientas de trabajo documental,
- y aplicaciones que necesiten consulta semántica + estructurada sin depender de tres sistemas separados.

2.2 Qué no es VantaDB en esta fase

VantaDB **no** debe presentarse todavía como:

- reemplazo total de una base OLTP empresarial completa,
- warehouse analítico,

- motor distribuido,
- graph database full-featured de propósito general,
- ni plataforma enterprise madura con HA, RBAC completo y cumplimiento formal.

2.3 Propuesta de valor correcta

Propuesta de valor inicial recomendada:

> “Memoria persistente embebida con búsqueda híbrida y filtros estructurados para agentes y aplicaciones de conocimiento, con mínima fricción operativa y un modelo de datos unificado.”

2.4 Propuesta de valor que debe evitarse

> “Una sola base de datos que reemplaza archivos, vectores, search engine, metadata store, graph DB y plataforma enterprise al mismo tiempo.”

Esa formulación es comercialmente atractiva, pero técnicamente prematura. Te obliga a prometer más de lo que la arquitectura actual puede sostener.

3. Diagnóstico del estado actual

3.1 Diagnóstico central

El problema principal del proyecto no es “RocksDB vs Fjall” en abstracto. El problema principal es **dispersión estratégica**.

Hoy el proyecto intenta ser:

- motor embebido tipo SQLite,
- plataforma server mode,
- memoria para agentes,
- base multimodelo general,
- runtime Python in-process,
- motor con API HTTP,
- y fundamento enterprise futuro.

Cada una de esas líneas es razonable por separado. Juntas, sin jerarquía clara, generan:

- roadmap confuso,
- deuda técnica temprana,
- mensajería de producto ambigua,
- y una arquitectura que corre el riesgo de resolver muchas cosas a medias.

3.2 Fortalezas observables

8. ****La tesis de mercado es buena.****

Hay dolor real en mantener embeddings, metadata, filtros, búsqueda textual y documentos en herramientas separadas.

9. ****El enfoque embebido/local-first tiene sentido.****

Reduce fricción de adopción y crea una diferencia clara frente a stacks más pesados.

10. ****Ya existe disciplina técnica útil.****

El repositorio evidencia intención de rigor: benchmarks, tests de recall, reglas de nomenclatura técnica y documentos de decisiones.

11. ****La visión unificada puede diferenciar el producto.****

Vectores + metadatos + relaciones + memoria persistente es una combinación con tracción real, si se ejecuta con contratos estrictos.

12. ****La narrativa “menos hardware, más simplicidad” sí puede ser competitiva.****

Pero solo si dejas de mezclarla con promesas enterprise de primera ola.

3.3 Debilidades actuales

13. ****Posicionamiento demasiado amplio.****

El producto se describe como memoria, motor vectorial, multimodelo, embedded, Python in-process, HTTP server y futura plataforma enterprise. Eso debilita el mensaje.

14. ****Acoplamiento conceptual excesivo.****

La idea de `UnifiedNode` es potente, pero corre el riesgo de convertirse en un objeto excesivamente cargado y difícil de evolucionar.

15. ****Superficie de dependencias demasiado amplia para la etapa actual.****

El stack ya incluye storage, servidor, Python, observabilidad, benchmarking y tooling pesado.

16. ****Empaquetado y renombrado incompletos.****

Persisten inconsistencias operativas que indican deuda de release engineering.

17. ****Telemetría dudosa de memoria.****

Si tus métricas de RAM son inconsistentes, no puedes tomar decisiones serias de performance y footprint.

3.4 Evidencia operativa ya detectada

- Dependencia actual fuerte de RocksDB.
- Pipeline de CI con swap adicional.
- Binario declarado como `vanta-server` mientras parte del empaquetado todavía compila `connectome-server`.
- Arquitectura pública que sigue mezclando embebido puro con modo servidor y sidecar de Ollama.
- Benchmarks de recall presentes, pero con falta de cierre sobre coherencia de mutaciones.
- Persistencia y reconstrucción descritas más por intención que por contratos cerrados.

4. Problema de mercado y oportunidad

4.1 Problema real a resolver

El usuario técnico actual, especialmente quien trabaja con agentes o herramientas de conocimiento, suele terminar con un stack fragmentado:

- archivos Markdown o documentos sueltos,
- embeddings en una vector store,
- metadatos en otra base,
- búsqueda textual en otro motor,
- y lógica de sincronización escrita a mano.

Eso produce:

- complejidad operativa,
- duplicación de datos,
- problemas de coherencia,

- más puntos de fallo,
- peor observabilidad,
- y más deuda de mantenimiento.

4.2 Oportunidad

La oportunidad de VantaDB está en ofrecer un núcleo único que reduzca esa fragmentación sin imponer complejidad de operación.

El proyecto puede capturar dos rutas de mercado:

Ruta A — Personal / developer / local-first

Usuario que quiere:

- memoria persistente,
- instalación simple,
- cero infraestructura,
- buen rendimiento local,
- APIs listas para agentes,
- exportación sencilla,
- y control directo sobre sus datos.

Ruta B — Equipos / empresa pequeña / enterprise futuro

Usuario que quiere:

- gobernanza,
- permisos,
- auditoría,
- límites por tenant,
- backups,
- recuperación,
- observabilidad,
- y una capa unificada en vez de tres motores distintos.

Conclusión: El núcleo puede ser común, pero el empaque del producto debe dividirse desde el diseño:

- ****VantaDB Personal****
- ****VantaDB Enterprise****

No como dos motores distintos, sino como dos presentaciones del mismo núcleo.

5. Segmentación y usuario objetivo

5.1 Segmento inicial recomendado

Segmento inicial prioritario:

- desarrolladores,
- builders de agentes,
- usuarios técnicos individuales,
- pequeñas herramientas internas,
- flujos de conocimiento local-first,
- asistentes de escritorio,

- copilotos privados,
- motores de memoria para aplicaciones pequeñas.

5.2 Segmento secundario

- equipos pequeños,
- empresas con prototipos internos,
- herramientas departamentales,
- casos de uso con bajo volumen inicial pero necesidad de trazabilidad.

5.3 Segmento que no debe ser la prioridad inicial

- enterprise regulado,
- operación distribuida,
- multi-tenant complejo,
- casos con HA obligatoria desde el día uno,
- cumplimiento formal,
- compras institucionales con exigencias de SLA y auditoría avanzada.

6. Visión del producto

6.1 Visión a 12-24 meses

Construir un núcleo embebido y coherente que sea:

- excelente como memoria persistente para agentes,
- competitivo como retrieval store híbrido,
- útil como base documental estructurada,
- y extensible hacia server mode y enterprise.

6.2 Visión inicial

El producto inicial debe ser:

> Un motor embebido de memoria persistente y retrieval híbrido que permita almacenar, versionar, filtrar y recuperar conocimiento con baja fricción y contratos de coherencia claros.

7. Principios rectores

18. ****La coherencia importa más que una demo rápida.****
19. ****El storage canónico debe estar separado de los índices derivados.****
20. ****El modo embedded es el núcleo; el modo server es un wrapper.****
21. ****No se añade otro motor; se añaden índices y planner.****
22. ****No se promete enterprise hasta tener durabilidad, recovery y observabilidad creíbles.****
23. ****Todo benchmark debe ir acompañado de semántica clara y métricas confiables.****
24. ****El MVP se gana por simplicidad operativa y consistencia, no por hype multimodelo.****

8. Objetivos del PRD

8.1 Objetivo principal

Definir un producto inicial viable, técnicamente coherente y comercialmente defendible para VantaDB.

8.2 Objetivos específicos

- Alinear visión, arquitectura y roadmap.
- Convertir los informes existentes en decisiones ejecutables.
- Establecer requerimientos funcionales y no funcionales.
- Fijar un roadmap concreto de 90 días.
- Priorizar un backlog realista.
- Reducir deuda técnica de base.
- Establecer criterios de salida del MVP.

9. No-objetivos

Durante la primera etapa, VantaDB no debe comprometerse con:

- clustering distribuido,
- consenso tipo Raft,
- data warehouse,
- SQL completo,
- graph query language full-featured,
- OLTP enterprise de propósito general,
- time-series especializada,
- replicación multi-región,
- marketplace de plugins,
- integración profunda con LLM runtime dentro del core,
- o dependencia operativa obligatoria de Ollama en el despliegue base.

10. Declaración de alcance

10.1 Dentro del alcance del MVP

- motor embebido/local-first,
- almacenamiento de documentos/fragmentos con metadata,
- embeddings,
- vector search,
- filtros estructurados,
- namespaces,
- persistencia durable,
- versionado básico,
- export/import,
- rebuild de índices,
- Python SDK simple,
- UI mínima o CLI fuerte,
- y un modelo de consulta unificado para retrieval básico.

10.2 Fuera del alcance del MVP

- multi-tenancy real,
- RBAC completo,
- auditoría enterprise,
- HA,

- replicación avanzada,
- despliegue cloud gestionado,
- lenguaje IQL como componente central del producto,
- traversal complejo de grafo como prioridad comercial,
- integración profunda con pandas/Arrow como promesa principal.

11. Requerimientos funcionales

11.1 API de memoria persistente

El producto debe ofrecer primitivas simples y consistentes:

- ``put(namespace, key, payload, metadata)``
- ``get(namespace, key)``
- ``delete(namespace, key)``
- ``search(namespace, query_vector, filters, text_query, top_k)``
- ``list(namespace, filters, pagination)``
- ``upsert_batch(...)``
- ``export(...)``
- ``import(...)``

Requisitos

- upsert idempotente por ``namespace + key``
- timestamps de creación y actualización
- control de versión del registro
- capacidad de sobrescribir o crear nueva revisión

11.2 Namespaces first-class

Los namespaces deben ser una primitiva de primera clase. No un adorno.

Ejemplos:

- ``user/<id>``
- ``org/<id>/workspace/<id>``
- ``agent/<id>/memory/<scope>``
- ``project/<id>``

Requisitos

- aislamiento lógico,
- búsqueda acotada al namespace,
- exportación selectiva por namespace,
- cuotas futuras por namespace/tenant.

11.3 Búsqueda vectorial

Requisitos mínimos:

- ANN con HNSW,
- top-k configurable,
- filtros estructurados,
- distancias soportadas claramente definidas,

- configuración de parámetros de índice,
- rebuild determinista del índice.

11.4 Búsqueda textual / léxica

Requisitos mínimos:

- índice invertido,
- BM25 o equivalente,
- tokenización configurable en lo básico,
- búsqueda por palabras clave,
- soporte de exact match razonable.

11.5 Búsqueda híbrida

Debe poder combinar:

- texto,
- vector,
- filtros,
- y ranking fusionado.

Requisitos

- estrategia de fusión explícita,
- RRF como primer candidato por simplicidad,
- score final interpretable,
- exposición del detalle del ranking cuando el modo debug esté activo.

11.6 Modelo documento + metadata

El dato canónico debe admitir:

- contenido textual,
- metadata tipada,
- embeddings,
- referencias lógicas,
- timestamps,
- y versión.

Requisitos

- esquema opcional, no obligatorio rígido,
- validación básica de tipos,
- campos indexables declarables,
- soporte futuro para constraints mínimas.

11.7 Versionado

Cada mutación importante debe poder:

- generar una nueva versión,
- o sobrescribir una versión actual con control explícito.

Requisitos

- `version\_id`
- `parent\_version\_id`
- política de latest version
- opción de recuperar versiones anteriores
- trazabilidad del origen de cambio

11.8 Ingesta

Pipeline mínimo:

25. lectura del archivo o payload
26. fingerprint/hash
27. deduplicación
28. normalización
29. chunking
30. generación de embeddings
31. indexación
32. confirmación de persistencia

11.9 Export / import

Necesario para:

- evitar lock-in,
- respaldo local,
- migración,
- reproducibilidad.

11.10 Herramientas de administración

Mínimos:

- stats de storage,
- rebuild de índices,
- compactación manual,
- verificación básica,
- snapshot,
- logs de eventos de mutación,
- inspección por namespace.

12. Requerimientos no funcionales

12.1 Simplicidad operativa

Para el producto inicial:

- instalación local simple,
- sin servidor obligatorio,
- sin dependencias de infraestructura pesada,
- experiencia tipo “single binary + local data dir”.

12.2 Durabilidad

Debe haber:

- WAL,
- checksums,
- replay,
- reconstrucción determinista de índices,
- política clara de snapshots,
- comportamiento documentado tras crash.

12.3 Observabilidad

El sistema debe exponer:

- uso real de RAM,
- tamaño en disco,
- tiempo de rebuild,
- latencia por consulta,
- throughput de ingesta,
- tamaño del WAL,
- estado de compaction,
- conteo de namespaces, documentos y chunks.

12.4 Performance

Métricas objetivo del MVP:

- primera consulta útil en menos de 5 minutos desde instalación
- top-k p95 razonable en datasets medianos
- rebuild ANN medible y no opaco
- ingesta incremental sin degradación explosiva

12.5 Portabilidad

Prioridad:

- Linux
- macOS
- Windows usable para desarrollo

12.6 Separación de modos

El core embebido no debe depender del server mode.

13. Casos de uso principales

13.1 Memoria persistente de un agente personal

Un agente guarda:

- preferencias,
- hechos recordables,
- resultados resumidos,
- snippets útiles,

- historial seleccionado,
- y conocimiento externo indexado.

El agente debe poder:

- leer memorias relevantes por namespace,
- filtrar por etiquetas,
- recuperar por búsqueda híbrida,
- y entender qué documento o evento originó cada memoria.

13.2 Sustitución de carpetas Markdown masivas

Un usuario técnico quiere:

- dejar de depender de cientos de `.md`,
- conservar control local,
- indexar notas, documentos, tickets, repos, snippets,
- y poder hacer búsqueda semántica + estructurada.

13.3 Base documental interna para un equipo pequeño

Una organización pequeña quiere:

- una base única para documentos, embeddings y metadata,
- sin mantener tres motores,
- con exportación y control razonable,
- y con posible evolución posterior a server mode.

14. Problemas actuales de arquitectura

14.1 Identidad dual del producto

Actualmente compiten dos ideas:

- producto embebido e in-process,
- producto servidor con HTTP, Docker y sidecars.

Esto debe resolverse así:

- ****embedded core primero****
- ****server wrapper después****

14.2 `UnifiedNode` como posible God Object

La abstracción es útil, pero peligrosa si se vuelve demasiado rígida y centraliza demasiadas responsabilidades:

- dato canónico,
- índices,
- metadata,
- relaciones,
- flags,
- optimizaciones de storage.

14.3 Falta de query planner real

Sin planner:

- los filtros se vuelven ad hoc,
- la búsqueda híbrida no escala bien,
- el producto no reemplaza realmente un stack fragmentado.

14.4 Persistencia e índices mal acotados

Todavía falta cerrar:

- qué vive como verdad canónica,
- qué es derivado,
- cuándo se reindexa,
- y qué pasa si el proceso cae entre la escritura del dato y la actualización del índice.

14.5 Mezcla de concerns

El proyecto mezcla:

- storage,
- retrieval,
- embeddings,
- Python runtime,
- HTTP,
- Docker,
- observabilidad,
- benchmarking,
- y estrategia de marketing.

Eso es demasiado acoplamiento para una fase temprana.

15. Riesgo dominante: coherencia

El riesgo técnico más importante del producto es devolver resultados inconsistentes.

Ejemplos de fallo:

- documento actualizado con embedding viejo,
- metadata nueva con índice textual viejo,
- ANN nuevo con payload index antiguo,
- versiones mezcladas entre latest y snapshot,
- relaciones no reconciliadas con el contenido,
- crash entre persistencia del dato y persistencia del índice.

15.1 Principio de diseño obligatorio

El dato canónico debe vivir en una representación base durable. Todos los índices y materializaciones deben ser derivados reconstruibles.

15.2 Implicaciones

- El índice vectorial no debe ser tratado como única fuente de verdad.
- El índice textual tampoco.

- Los payload indexes tampoco.
- Debe existir un protocolo de mutación.
- Debe existir un journal/WAL.
- Debe existir un estado de indexación por documento.

16. Capas clave que faltan

16.1 Capa de consulta unificada

Se necesita un contrato de consulta coherente que combine:

- vector query,
- text query,
- filters,
- scoring fusion,
- y opcionalmente referencias o relaciones.

16.2 Query planner

Debe decidir entre estrategias como:

- ANN con filtro,
- filtro primero y scan vectorial posterior,
- índice léxico primero,
- combinación híbrida con fusión.

16.3 Índice invertido / BM25

Sin esto no existe búsqueda híbrida real.

16.4 Protocolo de mutación

Debe definir:

- orden de escritura,
- estado del documento,
- indexación pendiente,
- reindexación,
- confirmación de consistencia,
- y comportamiento tras crash.

16.5 Instrumentación confiable

La observabilidad actual no es suficiente para dirigir el producto con confianza.

17. Complejidad real del multimodelo

Multimodelo no significa “guardar varios tipos de datos”. Significa sostener **consistencia, rendimiento y semántica de consulta entre ellos**.

Un sistema multimodelo serio exige:

- source of truth clara,
- índices especializados,
- planner,

- versionado,
- snapshots,
- recovery,
- reglas de mutación,
- y contratos de reconstrucción.

17.1 Problemas que hoy se están subestimando

- invalidación de embeddings tras cambios del documento,
- coordinación ANN + payload index + inverted index,
- costo de reconstrucción,
- page cache y mmap bajo presión real,
- consistencia de resultados bajo escritura concurrente,
- semántica de latest version,
- y costo real de mantener un “grafo + documento + vector + metadata” sin deuda estructural.

18. Storage: RocksDB vs Fjall

18.1 Estado real de RocksDB en este proyecto

RocksDB te da:

- madurez,
- tooling,
- historial industrial,
- base LSM robusta,
- features de backup/checkpoints,
- confianza operativa.

Pero hoy también te da:

- fricción de build,
- complejidad en Windows,
- dependencia C++,
- inner loop más lento,
- más complejidad de empaquetado,
- y más superficie de fallo.

18.2 Estado real de Fjall para este proyecto

Fjall representa:

- Rust puro,
- menor fricción de integración,
- mejor alineación con la filosofía embebida del proyecto,
- LSM nativo,
- y menor deuda operativa en esta etapa.

18.3 Decisión estratégica recomendada

Decisión

- ****Fjall como backend principal del core****

- ****RocksDB como backend opcional / benchmark / escape hatch****

Razón

El problema actual del proyecto no es falta de tooling enterprise. El problema es velocidad de ejecución y coherencia del producto.

RocksDB es mejor para ciertas cosas maduras, pero hoy te impone un costo que no estás convirtiendo en valor de producto.

18.4 Implicación técnica

Introducir una interfaz:

```
```text StorageBackend ```
```

Con implementaciones:

- `FjallBackend`
- `RocksDbBackend` (opcional)
- `InMemoryBackend` (tests / dev)

## 19. Persistencia y memoria

### 19.1 Principio recomendado

- Documento + metadata canónica + journal = fuente de verdad
- Índice ANN = derivado reconstruible
- Índice invertido = derivado reconstruible
- Payload indexes = derivados reconstruibles

### 19.2 Requisitos de durabilidad

33. WAL/journal
34. checksum por bloque o segmento
35. recovery determinista
36. snapshots consistentes
37. estrategia de rebuild de índices
38. pruebas de brutalidad:
  - kill -9
  - reinicio inesperado
  - caída durante indexación
  - caída durante compactación

### 19.3 Memoria

Necesitas diferenciar claramente:

- memoria del core,
- memoria del índice vectorial,
- page cache,
- mmap,
- buffers de ingesta,
- y caches temporales.

## 19.4 Problema actual

La telemetría existente sugiere que al menos parte de la medición actual no es confiable. Eso debe tratarse como incidente de plataforma, no como detalle menor.

## 20. Arquitectura objetivo recomendada

### 20.1 Capa 1 — Storage canónico

Responsabilidades:

- documentos
- metadata
- versionado
- WAL
- snapshots
- replay
- compaction policy

### 20.2 Capa 2 — Índices derivados

- ANN / HNSW
- inverted index / BM25
- payload indexes
- caches locales opcionales

### 20.3 Capa 3 — Query planner

- estrategia según filtros
- combinación text + vector
- fusión de scores
- optimización por selectividad

### 20.4 Capa 4 — Runtime embebido / SDK

- Python SDK
- CLI
- bindings futuros
- tool UX

### 20.5 Capa 5 — Server wrapper opcional

- HTTP API
- auth futura
- observabilidad
- multi-user posterior

### 20.6 Regla estructural

El **embedded core** no debe depender del modo servidor. El modo servidor envuelve al core. Nunca al revés.

## 21. Requerimientos del MVP

### 21.1 Requerimientos funcionales P0

- storage canónico durable
- namespaces
- put/get/delete
- vector search
- filtros estructurados
- modelo documento + metadata
- export/import
- rebuild ANN
- Python SDK usable
- CLI o UI mínima

### 21.2 Requerimientos funcionales P1

- índice invertido
- búsqueda híbrida
- versionado recuperable
- payload indexes
- inspección de índices y estado

### 21.3 Requerimientos funcionales P2

- planner adaptativo
- trazabilidad profunda
- UI local más rica
- server wrapper más sólido

## 22. KPI y métricas de éxito

### 22.1 KPIs de producto

- tiempo desde instalación hasta primera consulta útil
- número de pasos para indexar una fuente local
- tiempo de exportación/importación
- tiempo de reconstrucción del índice

### 22.2 KPIs técnicos

- p95/p99 de top-k con filtros
- throughput de ingesta por minuto
- tamaño en disco por millón de chunks
- tiempo de recovery tras crash
- recall sobre datasets representativos
- consumo real de RAM

### 22.3 Criterios de salida del MVP

El MVP está listo cuando:

- 39. el core compila y corre de forma estable en los entornos objetivo,
- 40. la memoria es persistente y recuperable,
- 41. existe búsqueda vectorial + filtros confiable,
- 42. namespaces funcionan,
- 43. la observabilidad básica es creíble,
- 44. la indexación se puede reconstruir,
- 45. el producto se puede instalar y usar sin infraestructura pesada,
- 46. y no depende de otro motor de búsqueda para cumplir la propuesta de valor básica.

## 23. Criterios de aceptación por área

### 23.1 Storage

- escritura durable confirmada
- recovery reproducible
- snapshots verificables

### 23.2 Query

- resultados consistentes
- filtros correctos
- top-k estable dentro de tolerancia

### 23.3 Embeddings e indexación

- reindexación controlada
- invalidación clara
- rebuild medible

### 23.4 Developer experience

- instalación simple
- errores accionables
- documentación honesta
- ejemplos funcionales

## 24. Roadmap de 90 días

### Fase 0 — Congelación estratégica (Días 1-7)

#### Objetivo

Congelar alcance, arquitectura inicial y decisiones no negociables.

#### Entregables

- este PRD aprobado
- definición formal del MVP
- decisión documentada sobre Fjall vs RocksDB
- definición del trait `StorageBackend`
- decisión formal sobre embedded-first

## Criterio de salida

- alcance cerrado
- no abrir nuevas líneas de producto
- backlog priorizado aprobado

## Fase 1 — Estabilización del núcleo (Días 8-30)

### Objetivos

- reducir fricción de build
- sanear release engineering
- separar concerns
- preparar migración o prototipo de backend

### Entregables

47. `StorageBackend` definido
48. perfil `dev-fast` sin LTO agresivo
49. corrección de naming y empaquetado
50. limpieza de inconsistencias `connectome-server`/`vanta-server`
51. medición correcta de memoria
52. crate/core aislado del modo server
53. backend Fjall prototipado detrás de interfaz
54. backend RocksDB mantenido opcional

### Riesgos mitigados

- inner loop lento
- build frágil
- identidad dual mal resuelta
- deuda de renombrado
- telemetría no confiable

## Criterio de salida

- build reproducible
- core aislado
- observabilidad básica confiable
- decisión de backend aterrizada en código

## Fase 2 — MVP funcional (Días 31-60)

### Objetivos

- entregar memoria persistente usable
- cerrar modelo canónico
- hacer retrieval confiable

### Entregables

55. modelo documento + metadata canónica

- 56. namespaces first-class
- 57. put/get/delete/list/export/import
- 58. vector search + filtros
- 59. journal/WAL
- 60. recovery y replay
- 61. rebuild ANN
- 62. Python SDK mínimo usable
- 63. CLI o UI local básica
- 64. documentación operativa honesta

## Riesgos mitigados

- producto aún abstracto
- falta de flujo usable
- falta de persistencia real
- semántica difusa de memoria

## Criterio de salida

- un usuario técnico puede instalar, indexar, buscar y reiniciar sin perder coherencia

## Fase 3 — Diferenciación inicial (Días 61-90)

### Objetivos

- incorporar hybrid search real
- preparar v1
- cerrar backlog P0 y parte de P1

### Entregables

- 65. inverted index mínimo
- 66. BM25 o equivalente
- 67. búsqueda híbrida con RRF
- 68. payload indexes
- 69. protocolo de mutación documentado
- 70. versionado básico recuperable
- 71. métricas p95/p99
- 72. suite de pruebas de brutalidad
- 73. benchmark reproducible del flujo de memoria
- 74. decisión formal del wrapper server para la siguiente etapa

### Criterio de salida

- el producto ya no es solo una base vectorial embebida;
- es una memoria persistente híbrida y administrable.

## 25. Backlog priorizado

### 25.1 P0 — No negociables

ID	Área	Item	Problema que resuelve	Criterio de aceptación	Esfuerzo
P0-01	Producto	Congelar definición del MVP	Evita dispersión estratégica	Documento aprobado y scope cerrado	S
P0-02	Arquitectura	Introducir `StorageBackend`	Desacopla storage del backend actual	Trait compilando + implementación stub	M
P0-03	Build	Crear perfil `dev-fast`	Reduce fricción del inner loop	Build local claramente más rápido	S
P0-04	Build	Corregir binarios/naming	Evita errores de release	Dockerfile, scripts y CI alineados	M
P0-05	Observabilidad	Corregir telemetría de memoria	Sin métricas confiables no hay tuning serio	RAM reportada consistente con host real	M
P0-06	Arquitectura	Separar `core` del modo servidor	Reduce acoplamiento y build	Crate core compila sin server deps	L
P0-07	Storage	Prototipo Fjall detrás de interfaz	Permite validar la decisión estratégica	Core usable con backend Fjall	L
P0-08	Storage	Mantener RocksDB opcional	Conserva escape hatch	Feature flag / backend opcional funcional	M
P0-09	Data model	Definir modelo canónico	Evita incoherencia futura	Spec cerrada y tests de persistencia	M
P0-10	Query	Namespaces first-class	Necesario para memoria útil	CRUD y search por namespace	M
P0-11	Query	Put/Get/Delete/List	API mínima de memoria	SDK/CLI funcionando	M
P0-12	Retrieval	Vector search + filtros	Caso de uso central	top-k + filtros correcto y testeado	L
P0-13	Durabilidad	WAL y recovery base	Sin esto no hay memoria persistente	crash test básico pasa	L
P0-14	Indexing	Rebuild ANN determinista	Evita dependencia tóxica del índice	comando o función de rebuild disponible	M

P0-15	DX	Export/Import	Evita lock-in y mejora adopción	Export e import round-trip pasan	M
P0-16	SDK	Python SDK mínimo usable	Segmento inicial depende de esto	flujo simple documentado y funcional	L
P0-17	QA	Suite kill/restart/recovery	Valida durabilidad real	pruebas automatizadas pasan	M
P0-18	Docs	README honesto y nuevo quickstart	Alinea narrativa y realidad	guía funcional y verificable	S

## 25.2 P1 — Muy importantes

ID	Área	Item	Problema que resuelve	Criterio de aceptación	Esfuerzo
P1-01	Retrieval	Índice invertido mínimo	Sin esto no hay híbrida real	búsqueda keyword funcional	L
P1-02	Retrieval	BM25 o equivalente	Relevancia léxica útil	ranking básico validado	M
P1-03	Retrieval	Fusión RRF	Une vector + texto	hybrid search con scoring consistente	M
P1-04	Indexing	Payload indexes	Mejora filtros y evita scans torpes	campos indexables configurables	M
P1-05	Versionado	Versionado recuperable	Evita sobrescritura opaca	latest + historic retrieval	M
P1-06	Mutaciones	Protocolo de mutación documentado	Reduce riesgo de incoherencia	secuencia de escritura definida y testeada	M
P1-07	Query	Query contract unificado	Evita APIs desconectadas	search() combinando vector/text/filter	M
P1-08	UI/CLI	Inspector local	Reduce opacidad para el usuario	listado, filtros, stats locales	M
P1-09	Metrics	p95/p99 y throughput	Permite decisiones reales	métricas expuestas y confiables	M
P1-10	QA	Benchmarks reproducibles del flujo real	Evita benchmarks irrelevantes	benchmark documentado con datos reales	M

## 25.3 P2 — Posteriores

ID	Área	Item	Problema que resuelve	Criterio de aceptación	Esfuerzo
P2-01	Query	Planner adaptativo	Optimiza estrategias de búsqueda	planner simple por selectividad	L
P2-02	Server	Wrapper HTTP limpio	Abre ruta a equipos	server separado del core	M
P2-03	Security	Auth básica del wrapper	Prepara modo compartido	auth mínima documentada	M
P2-04	Enterprise	Multi-tenancy inicial	Abre ruta B2B	aislamiento lógico por tenant	L
P2-05	Enterprise	Audit log	Requisito de gobernanza	log de mutaciones y accesos	M
P2-06	Enterprise	RBAC básico	Control de uso	roles mínimos funcionales	L
P2-07	Ops	Snapshots administrables	Mejora operación	snapshot/resto re probado	M
P2-08	Ops	Encryption	Requisito futuro	cifrado configurable	L

## 26. Historias de usuario prioritarias

### Historia 1

Como desarrollador de agentes, quiero guardar memoria persistente por namespace para que mi agente recuerde contexto entre sesiones.

### Historia 2

Como usuario técnico individual, quiero indexar mis notas y documentos locales sin montar infraestructura.

### Historia 3

Como builder de herramientas, quiero combinar búsqueda semántica y filtros estructurados en una sola consulta.

### Historia 4

Como equipo pequeño, quiero exportar e importar la memoria para evitar lock-in y simplificar respaldos.

### Historia 5

Como operador del sistema, quiero reconstruir índices derivados sin comprometer la integridad del dato base.

## 27. Riesgos y mitigaciones

### Riesgo 1 — Dispersión estratégica

**Mitigación:** scope freeze, PRD aprobado, backlog cerrado.

### Riesgo 2 — Deuda de build

**Mitigación:** dev-fast profile, separación de crates, Fjall como backend principal.

### Riesgo 3 — Incoherencia entre representaciones

**Mitigación:** modelo canónico + protocolo de mutación + journal + rebuild.

### Riesgo 4 — Product messaging ambiguo

**Mitigación:** nuevo posicionamiento, README honesto, no vender enterprise todavía.

### Riesgo 5 — Benchmarks irrelevantes

**Mitigación:** medir flujos reales, no solo microbenchmarks.

### Riesgo 6 — Persistencia frágil

**Mitigación:** crash tests, replay, snapshots, rebuild.

### Riesgo 7 — HNSW persistido como verdad

**Mitigación:** tratar HNSW como materialización reconstruible.

## 28. Recomendaciones ejecutivas finales

75. \*\*Reduce el producto inicial a memoria persistente embebida + retrieval híbrido.\*\*
76. \*\*Congela la expansión enterprise como segunda ola.\*\*
77. \*\*Migra el core hacia Fjall.\*\*
78. \*\*Mantén RocksDB como opción, no como mandato del core.\*\*
79. \*\*No añadas otra base de datos.\*\*
80. \*\*Añade índice invertido, planner y protocolo de coherencia.\*\*
81. \*\*Separa embedded core de server mode.\*\*
82. \*\*Corrige build, naming y telemetría antes de ampliar features.\*\*
83. \*\*Haz que el dato canónico mande; los índices obedecen.\*\*
84. \*\*Mide todo lo importante y deja de discutir arquitectura sin métricas confiables.\*\*

## 29. Decisiones que no conviene postergar

Decisión	Recomendación
Segmento inicial	Developer / local-first
Modelo de producto	Memoria persistente + retrieval híbrido
Backend principal	Fjall
Backend secundario	RocksDB opcional
Índice adicional	Invertido/BM25
Fuente de verdad	Documento + metadata + journal
Índice ANN	Materialización reconstruible
Modo servidor	Wrapper separado, no núcleo

Prioridad de roadmap	Estabilización → MVP → híbrida → server/enterprise
Criterio de éxito	Instalación simple, coherencia dura, recuperación confiable

## 30. Cierre

VantaDB sí puede convertirse en una propuesta fuerte. Pero no por intentar hacer más desde ya.

Va a ganar si hace tres cosas mejor que otros:

- simplificar la operación,
- mantener coherencia entre representaciones,
- y ofrecer una memoria persistente híbrida realmente usable en local.

Tu batalla ahora no es contra la competencia. Es contra tu propia dispersión.

Si cierras mejor el producto, la arquitectura y el backlog, el proyecto deja de ser una tesis interesante y se convierte en un producto construible.